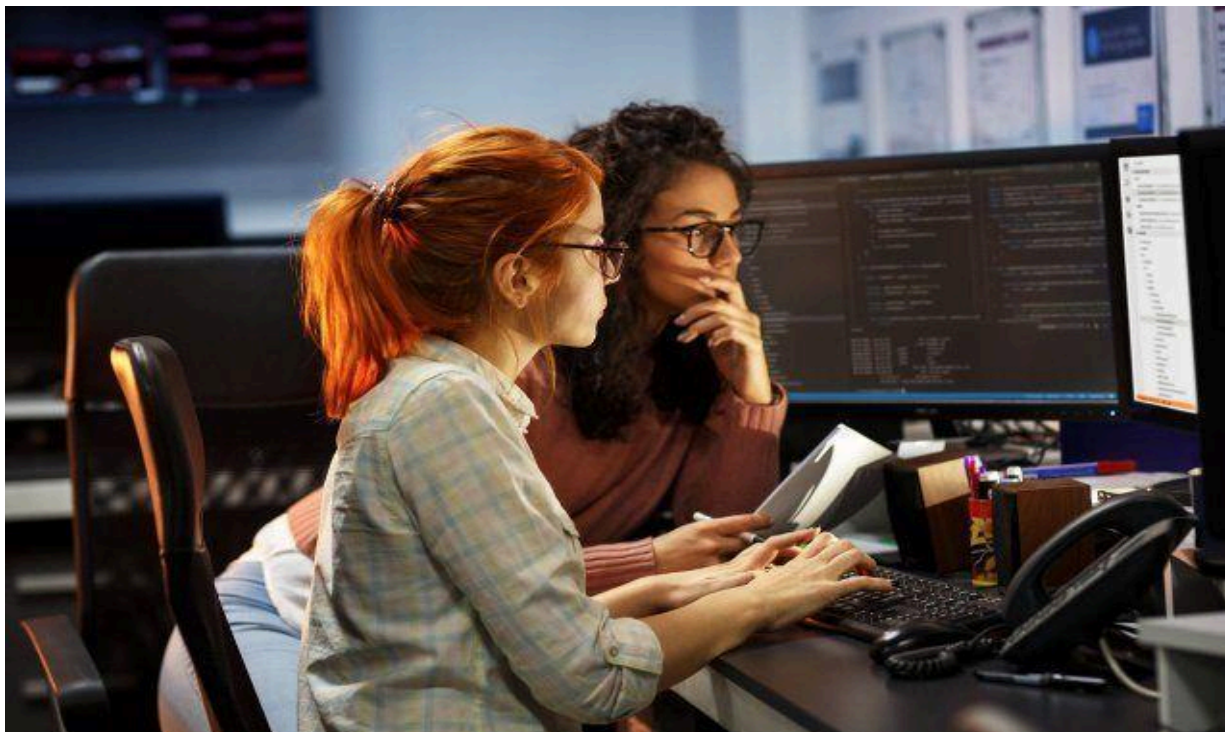




Universidad de El Salvador

Facultad Multidisciplinaria de Occidente – Ingeniería en Desarrollo de Software

Asignatura:
Programación Orientada a Objetos

Unidad #: 4

Tema: Persistencia, Archivos y Concurrencia Moderna

SEMANA 15

Autor:

Introducción:

En este material estudiaremos cinco conceptos fundamentales para comprender cómo funcionan las aplicaciones modernas basadas en sistemas escalables y arquitecturas distribuidas. Iniciaremos analizando **las limitaciones del modelo clásico de programación bloqueante**, donde las aplicaciones deben esperar a que finalice cada operación antes de continuar, generando cuellos de botella y bajo rendimiento.

A continuación exploraremos **el paradigma reactivo**, un enfoque que permite que las aplicaciones trabajen de forma asíncrona, no bloqueante y orientada al flujo de datos. Veremos cómo esta filosofía se aplica en Java a través de **Spring WebFlux y Project Reactor**, herramientas diseñadas para manejar grandes cantidades de solicitudes con menos recursos y mayor eficiencia.

Luego examinaremos la evolución natural hacia arquitecturas modernas con la transición **del monolito hacia los microservicios**, entendiendo por qué esta estructura mejora la escalabilidad, la modularidad y la resiliencia de los sistemas.

Finalmente, estudiaremos cómo se coordinan y gestionan estos microservicios mediante **Spring Cloud**, una plataforma que proporciona componentes clave para el descubrimiento de servicios, comunicación, enrutamiento, equilibrio de carga y tolerancia a fallos.

Semana 15

5. Introducción a la Programación Reactiva y Spring Cloud

5.1 El Problema: Las Limitaciones del Modelo Clásico (Blocking)

Concepto Clave

El modelo blocking (bloqueante) tradicional mantiene un hilo esperando mientras se completa una operación I/O, desperdiciando recursos.

● Problemas del Modelo Blocking

1. Bloqueo de Hilos

- Un hilo = Una petición
- El hilo queda BLOQUEADO esperando respuestas (DB, APIs, archivos)
- Recursos desperdiciados durante la espera

2. Escalabilidad Limitada

- Número finito de hilos disponibles
- Bajo alta carga: hilos agotados = peticiones rechazadas
- Más usuarios = Más hilos = Más memoria

3. Rendimiento Deficiente

- Context switching costoso
- Muchos hilos inactivos consumiendo memoria
- Latencias acumulativas

BLOCKING (Tradicional)

Thread 1: [Solicitud] → [Esperando DB] → [Responsta]
 Thread 2: [Request] → [Esperando API] → [Response]
 Thread 3: [Request] → [Esperando E/S] → [Respuesta]

- ✗ Hilos bloqueados esperando
- ✗ Recursos desperdiciados

REACTIVO (Non-Blocking)

Thread 1: [Req1] → DB → API → [Req3] → I/O
 [Response1] callback callback [Response2] [Response3]

- ✓ Un hilo maneja múltiples requests
- ✓ No bloqueos, callbacks/eventos

```
java
// ✗ BLOCKING - El hilo espera
@GetMapping("/user/{id}")
public User getUser(@PathVariable Long id) {
    User user = userRepository.findById(id); // BLOQUEA
    Orders orders = orderService.getOrders(id); // BLOQUEA
    return user;
}
```

Problema: Si la DB tarda 2 segundos, el hilo está **bloqueado** 2 segundos.

Limitaciones Principales

Aspecto	Limitación
Concurrencia	1 hilo = 1 request simultánea
Throughput	Limitado por pool de hilos
Latencia	Tiempos de espera acumulativos
Recursos	Alto consumo de memoria (muchos hilos)

Cuándo el Blocking es Problemático

- X Alta concurrencia (miles de usuarios)
- X Operaciones I/O lentas (APIs externas, DB)
- X Microservicios con muchas llamadas en cascada
- X Aplicaciones de streaming/tiempo real

Señales de que Blocking es un Problema

- ⚠ CPU baja (< 30%) pero sistema lento
- ⚠ Threads en estado WAITING/TIMED_WAITING
- ⚠ Timeouts frecuentes bajo carga
- ⚠ Memory OutOfMemory con muchos threads
- ⚠ Response time aumenta linealmente con carga

Solución: Programación Reactiva

Principio: Non-Blocking I/O

- Liberar threads durante esperas
- Callbacks/Eventos en lugar de bloqueos
- Pocos threads, alta concurrencia
- Frameworks: **Project Reactor**, **RxJava**, **Spring WebFlux**

Preview:

```
java
// ✅ NON-BLOCKING - Thread no espera
@GetMapping("/user/{id}")
public Mono<User> getUser(@PathVariable Long id) {
    return userRepository.findById(id); // Thread liberado
}
```

5.2 La Solución: El Paradigma Reactivo

Concepto Fundamental

La **Programación Reactiva** es un paradigma basado en **flujos de datos asíncronos** y **propagación de cambios**, donde las operaciones NO bloquean hilos, permitiendo alta concurrencia con pocos recursos.

Definición

Programación Reactiva = Asíncrono + Non-Blocking + Eventos + Streams

Modelo Mental:

- Los datos fluyen como un río (stream)
- Reaccionas a eventos cuando ocurren
- No esperas activamente (no blocking)
- Propagación automática de cambios

Principios Fundamentales

1. Asíncrono (Asynchronous)

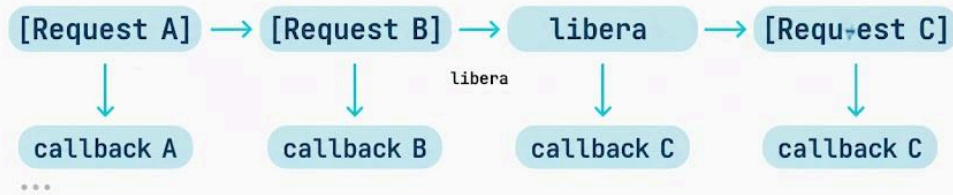
Las operaciones NO esperan resultados inmediatos, continúan ejecutando otras tareas.

```
// ✗ SÍNCRONO
String result = database.query(); // Espera aquí
process(result);

// ✔ ASÍNCRONO
database.query()
    .subscribe(result -> process(result)); // Continúa inmediatamente
...
```

2. Non-Blocking I/O Los hilos NO se bloquean esperando I/O, se liberan para otras tareas.

Thread Timeline:



3. Event-Driven (Basado en Eventos) El sistema reacciona a eventos (datos disponibles, error, completado).

Eventos:

- onNext(data) → Nuevo dato disponible
- onError(error) → Error ocurrido
- onComplete() → Stream terminado

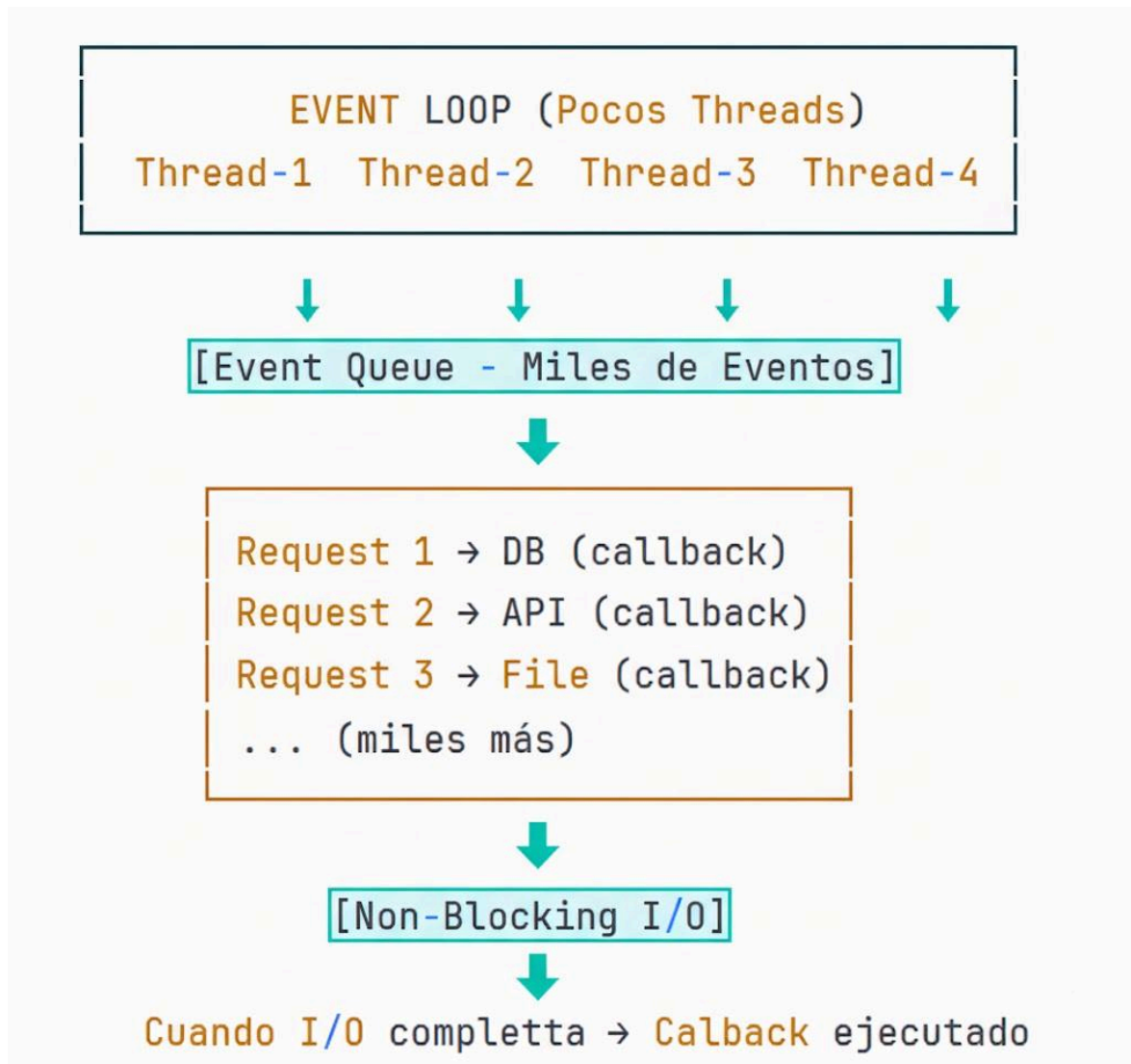
4. Backpressure

El consumidor controla la la velocidad de datos del productor.



Arquitectura Reactiva

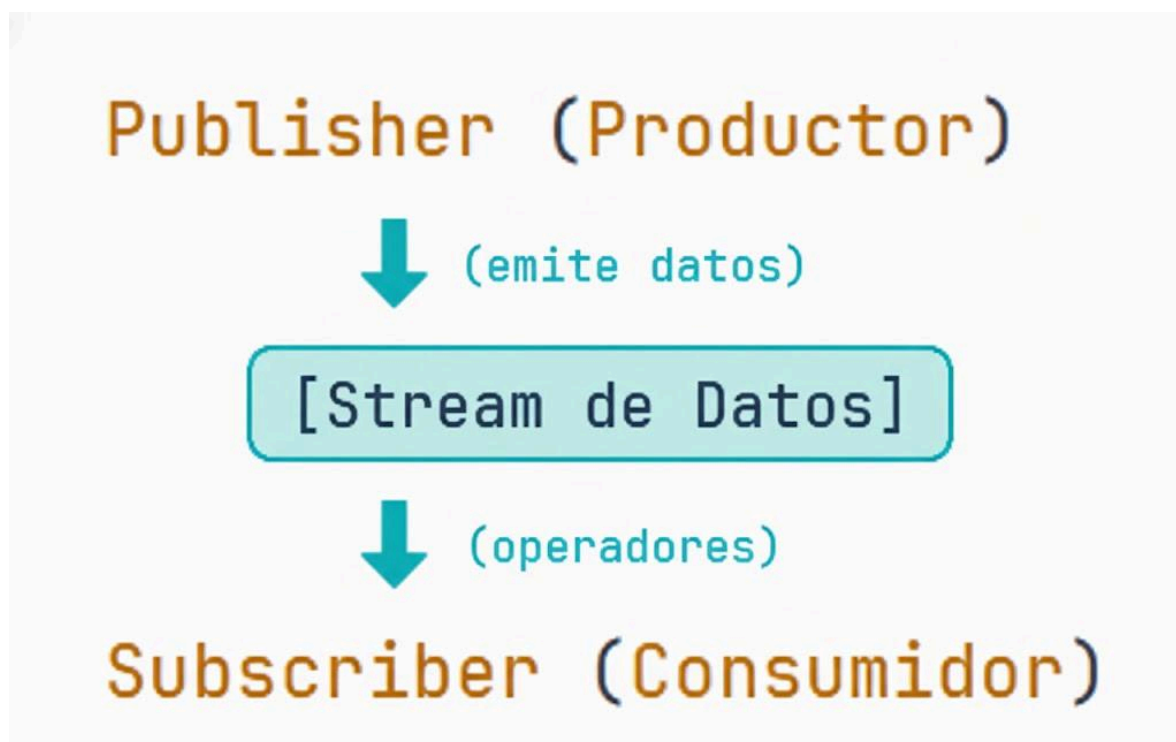
Modelo de Funcionamiento



4 threads manejan 10,000+ requests simultáneas

Flujo de Datos Reactivo (Reactive Streams)

Componentes Principales



Ejemplo Conceptual

```
java
// Publisher emite datos
Flux<Integer> numbers = Flux.just(1, 2, 3, 4, 5);

// Operadores transforman datos
numbers
    .filter(n -> n % 2 == 0)    // Solo pares
    .map(n -> n * 10)           // Multiplicar por 10
    .subscribe(result ->      // Consumir
        System.out.println(result)
    );

// Output: 20, 40 (non-blocking)
```


Reactive Streams API (Especificación)

```
java

// 1. Publisher - Produce datos
public interface Publisher<T> {
    void subscribe(Subscriber<? super T> s);
}

// 2. Subscriber - Consume datos
public interface Subscriber<T> {
    void onNext(T t);           // Nuevo dato
    void onError(Throwable t);  // Error
    void onComplete();          // Completado
    void onSubscribe(Subscription s);
}

// 3. Subscription - Control de flujo
public interface Subscription {
    void request(long n);       // Pedir n elementos
    void cancel();              // Cancelar
}

// 4. Processor - Procesa y transforma
public interface Processor<T, R>
    extends Subscriber<T>, Publisher<R> {
}
```

Project Reactor (Spring)

Tipos Reactivos Principales

Mono<T> - 0 o 1 elemento

```
java
// Representa un único valor o vacío
Mono<User> user = userRepository.findById(1L);

Mono<String> empty = Mono.empty(); // Sin valor
Mono<String> single = Mono.just("Hello"); // Un valor
```

Flux<T> - 0 a N elementos

```
java
// Representa un stream de múltiples valores
Flux<Product> products = productRepository.findAll();

Flux<Integer> range = Flux.range(1, 100); // 1 a 100
Flux<String> list = Flux.just("A", "B", "C");
```

Comparación

Tipo	Elementos	Uso Típico
Mono	0 o 1	Respuestas únicas (GET by ID)
Flux	0 a N	Listas, streams, eventos

Spring WebFlux vs Spring MVC

Aspecto	Spring MVC	Spring WebFlux
Modelo	Blocking	Non-Blocking
API	Servlet API	Reactive Streams
Servidor	Tomcat, Jetty	Netty, Undertow
Return Type	User, List<User>	Mono<User>, Flux<User>
Database	JDBC (blocking)	R2DBC (reactive)
HTTP Client	RestTemplate	WebClient

Cuándo Usar Reactivo

✓ Casos Ideales

- Alta concurrencia (miles de usuarios)
- Operaciones I/O intensivas
- Microservicios con muchas llamadas
- Streaming de datos en tiempo real
- APIs con latencias variables
- Aplicaciones con recursos limitados

✗ Cuándo NO Usar

- Operaciones CPU-intensive
- CRUD simple con baja carga
- Equipo sin experiencia reactiva
- Bases de datos sin soporte reactivo
- Aplicaciones legacy (migración costosa)

Ejemplo Completo: REST API Reactivo

```
@RestController
@RequestMapping("/api/products")
public class ProductController {

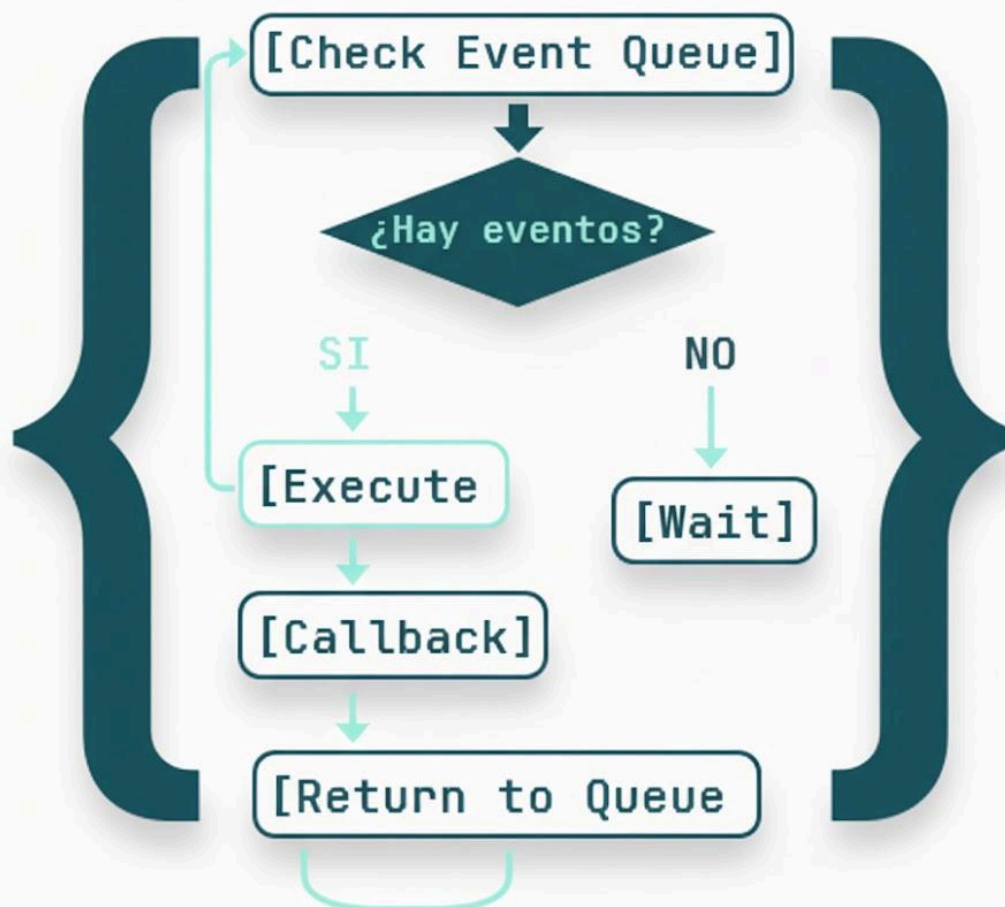
    @GetMapping("/{id}")
    public Mono<Product> getProduct(@PathVariable Long id) {
        return productService.findById(id)
            .switchIfEmpty(Mono.error(
                new NotFoundException("Product not found")
            ))
            .timeout(Duration.ofSeconds(5));
    }

    @GetMapping
    public Flux<Product> getAllProducts() {
        return productService.findAll()
            .take(100); // Limitar a 100
    }

    @PostMapping
    public Mono<Product> createProduct(@RequestBody Product product) {
        return productService.save(product);
    }
}
```

🔄 Event Loop Explicado

Event Loop (Similar a Node.js):



4 threads ejecutan este loop → manejan miles de eventos

Desafíos del Modelo Reactivo

1. Curva de Aprendizaje

- Pensamiento diferente (asíncrono)
- Debugging más complejo
- Stack traces confusos

2. Composición Compleja

```
java
// Puede volverse difícil de leer
flux.flatMap(x ->
    service1.call(x)
        .flatMap(y ->
            service2.call(y)
                .map(z -> transform(z))
        )
).subscribe();
```

3. No Mezclar Blocking con Reactivo

```
java
// ✗ MAL - Bloquea el pipeline reactivo
Mono.fromCallable(() -> {
    return blockingDatabaseCall(); // NUNCA hacer esto
})
```

Conceptos Clave para Recordar

1. **Lazy Evaluation** - No se ejecuta hasta `.subscribe()`
2. **Immutability** - Operadores crean nuevos streams
3. **Declarativo** - Describes QUÉ, no CÓMO
4. **Composable** - Encadenar operaciones
5. **Non-blocking** - Hilos nunca esperan

5.3 Programación Reactiva en Java: Spring WebFlux y Project Reactor

¿Qué es Spring WebFlux?

Spring WebFlux es el stack reactivo de Spring para construir aplicaciones web non-blocking basadas en **Project Reactor** y la especificación **Reactive Streams**.



Spring WebFlux = [Framework Web] + [Non-Blocking] + [Reactive Streams]
= Framework Web + Reactive Streams

Project Reactor: Tipos Principales

Mono<T>

Representa un flujo que emite **0 o 1 elemento**.

Casos de uso:

- Respuestas HTTP únicas
- Consultas de DB por ID
- Operaciones que retornan un único resultado

java

```
Mono<User> user = userRepository.findById(1L);  
Mono<String> value = Mono.just("Hello");  
Mono<Void> empty = Mono.empty();
```


Flux<T>

Representa un flujo que emite **0 a N elementos**.

Casos de uso:

- Listas de resultados
- Streams de datos
- Eventos en tiempo real

Flux<T>

Representa un flujo que emite **0 a N elementos**.

Casos de uso:

- Listas de resultados
- Streams de datos
- Eventos en tiempo real

java

```
Flux<Product> products = productRepository.findAll();  
Flux<Integer> numbers = Flux.range(1, 100);  
Flux<String> items = Flux.just("A", "B", "C");  
...
```

Cold vs Hot Publishers

- Emiten datos solo cuando hay subscriber
- Cada subscriber recibe la secuencia completa desde el inicio
- Unicast: Un stream por subscriber
- Analogía Netflix (cada usuario ve desde el inicio)

Ejemplos:

Llamadas HTTP, queries DB, archivos

Hot Publishers (Calientes)

- Emiten datos independiente de subscribers
- Subscribers reciben datos desde que se suscriben
- Multicast: Múltiples subscribers comparten stream
- Analogía: TV en vivo (te unes a transmisión en curso)

Ejemplos:

Event streams, WebSockets, Server-Sent Events

Cold: Cada subscriber → Nueva secuencia completa

Hot: Todos subscribers → Misma secuencia en curso

Schedulers: Control de Threading

Los **Schedulers** definen **en qué threads** se ejecuta el código reactivo.

Tipos Principales

Scheduler	Características	Uso
<code>immediate()</code>	Thread actual, sin cambio	Operaciones rápidas
<code>single()</code>	Un solo thread compartido	Tareas secuenciales
<code>parallel()</code>	Pool fijo (CPU cores)	CPU-intensive
<code>boundedElastic()</code>	Pool elástico con límite	Blocking inevitable (DB legacy, File I/O)

subscribeOn() vs **publishOn()**

subscribeOn(scheduler)

- Define dónde ocurre la **suscripción**
- Afecta todo el pipeline **upstream**
- Solo el **primero** tiene efecto

publishOn(scheduler)

- Define dónde se **procesan señales**
- Afecta todo **downstream** desde ese punto
- Pueden haber **múltiples** en pipeline

java

```
flux.subscribeOn(Schedulers.boundedElastic()) // Suscripción
      .map(x -> transform(x))                  // En boundedElastic
      .publishOn(Schedulers.parallel())         // Cambia a parallel
      .filter(x -> x > 10)                      // En parallel
      .subscribe();
```

Backpressure: Control de Flujo

Backpressure= Mecanismo para que el **consumidor**** controle la velocidad del ****productor****.**

El Problema

Productor:

1000 items/s Consumidor: 100 items/s ¿Qué hacer con 900 items/s restantes?

Estrategia	Acción	Pérdida de Datos	Uso
BUFFER	Almacenar en memoria	No	Ráfagas temporales
DROP	Descartar exceso	Sí	Métricas, datos no críticos
LATEST	Solo el último	Sí	Estado actual (precios, UI)
ERROR	Fallar stream	No	Datos críticos

```
flux.onBackpressureBuffer(100); // Almacenar max 100  
flux.onBackpressureDrop();      // Descartar exceso  
flux.onBackpressureLatest();    // Solo último  
flux.onBackpressureError();     // Error si no puede procesar
```

WebClient: Cliente HTTP Reactivo

WebClient reemplaza a **RestTemplate** en el mundo reactivo (non-blocking).

Configuración

java

```
WebClient client = WebClient.builder()  
    .baseUrl("https://api.example.com")  
    .defaultHeader("Content-Type", "application/json")  
    .build();
```

Operaciones Básicas

```
java

// GET
Mono<User> user = client.get()
    .uri("/users/{id}", 1)
    .retrieve()
    .bodyToMono(User.class);

// POST
Mono<User> created = client.post()
    .uri("/users")
    .bodyValue(newUser)
    .retrieve()
    .bodyToMono(User.class);
```

Ventajas sobre RestTemplate:

- Non-blocking
- Manejo reactivo de errores
- Streaming support
- Composable

R2DBC: Base de Datos Reactiva

R2DBC (Reactive Relational Database Connectivity) es la alternativa reactiva a JDBC.

Características

- **Non-blocking** a nivel de driver
- Compatible con: PostgreSQL, MySQL, H2, SQL Server
- Integración con Spring Data

Repository Reactivo

```
public interface UserRepository
    extends ReactiveCrudRepository<User, Long> {

    Flux<User> findByName(String name);
    Mono<User> findByEmail(String email);
}
```

Service Layer

```
java

@Service
public class UserService {

    public Mono<User> findById(Long id) {
        return userRepository.findById(id)
            .switchIfEmpty(Mono.error(
                new NotFoundException("User not found")
            ));
    }
}
```

JDBC vs R2DBC:

- JDBC: Blocking, thread-per-connection
- R2DBC: Non-blocking, alta concurrencia

Operadores Principales

Transformación

```
java

.map(x -> x * 2)           // Transformar síncronamente
.flatMap(x -> service(x))  // Transformar asíncronamente
.filter(x -> x > 10)       // Filtrar elementos
```

Combinación

```
java

Mono.zip(mono1, mono2)      // Combinar resultados
Flux.merge(flux1, flux2)    // Intercalar (paralelo)
Flux.concat(flux1, flux2)   // Secuencial
```

Manejo de Errores

```
java

.onErrorReturn(defaultValue) // Valor por defecto
.onErrorResume(e -> fallback()) // Fuente alternativa
.retry(3)                    // Reintentar 3 veces
.timeout(Duration.ofSeconds(5)) // Timeout máximo
```

Limitación

```
java

.take(10)      // Primeros 10 elementos
.skip(5)       // Saltar primeros 5
.takeLast(3)   // Últimos 3
.distinct()    // Elementos únicos
```

Context: Propagación de Datos

Reactor Context reemplaza a **ThreadLocal** en entornos reactivos.

¿Por qué no ThreadLocal?

- Operadores cambian de thread
- Context se pierde entre threads
- No funciona con async

Context Reactivo

- Map **immutable** de datos
- Propagado automáticamente en pipeline
- Thread-safe
- Disponible en cualquier operador

```
java

// Escribir en context
mono.contextWrite(Context.of("userId", "123"));

// Leer del context
mono.deferContextual(ctx -> {
    String userId = ctx.get("userId");
    return processWithUser(userId);
});
```

Usos comunes:

- Autenticación
- Request ID / Tracing
- Tenant ID (multi-tenancy)

Comparativa: MVC vs WebFlux

Aspecto	Spring MVC	Spring WebFlux
Modelo	Blocking, síncrono	Non-blocking, asíncrono
API	Servlet API	Reactive Streams
Threading	Thread-per-request	Event Loop
Servidor	Tomcat, Jetty	Netty (default)
DB	JDBC	R2DBC
HTTP Client	RestTemplate	WebClient
Throughput	~500 req/s	~15,000 req/s
Aprendizaje	Fácil	Complejo

Cuándo Usar WebFlux

✓ Usar WebFlux cuando:

- Alta concurrencia (miles de usuarios)
- I/O intensive (muchas llamadas externas)
- Microservicios con dependencias
- Streaming de datos
- Recursos limitados (optimizar costos)

✗ NO usar WebFlux cuando:

- CRUD simple, baja carga
- CPU-intensive operations
- Equipo sin experiencia reactiva
- Legacy systems sin R2DBC
- Prototipado rápido



Mejores Prácticas

1. **No mezclar blocking con reactive**
 - Nunca usar `.block()` en producción
 - No usar JDBC en WebFlux (usar R2DBC)
2. **Siempre manejar errores**
 - `onErrorResume()` para fallbacks
 - `timeout()` en llamadas externas
 - Circuit breakers (Resilience4j)

3. Testing completo

- Usar StepVerifier
- Test con tiempo virtual
- Test escenarios de error

4. Schedulers apropiados

- Usar `boundedElastic()` solo para blocking inevitable
- Event Loop para operaciones reactivas

5. Context en lugar de ThreadLocal

- Para datos de seguridad, tracing, etc.

Errores Comunes

Error	Problema	Solución
No subscribirse	Publisher no ejecuta	Siempre <code>.subscribe()</code> o <code>return</code>
Usar <code>.block()</code>	Bloquea thread reactivo	Mantener flujo reactivo
ThreadLocal	Se pierde en threads	Usar Reactor Context
Mixing blocking	JDBC en WebFlux	Usar R2DBC o <code>boundedElastic</code>

Resumen: Spring WebFlux + Project Reactor permiten construir aplicaciones altamente escalables con modelo non-blocking, utilizando Mono/Flux para representar flujos de datos asíncronos, Schedulers para control de threading, y WebClient/R2DBC para I/O reactivo.

5.4 El Siguiente Nivel: De Monolito a Microservicios

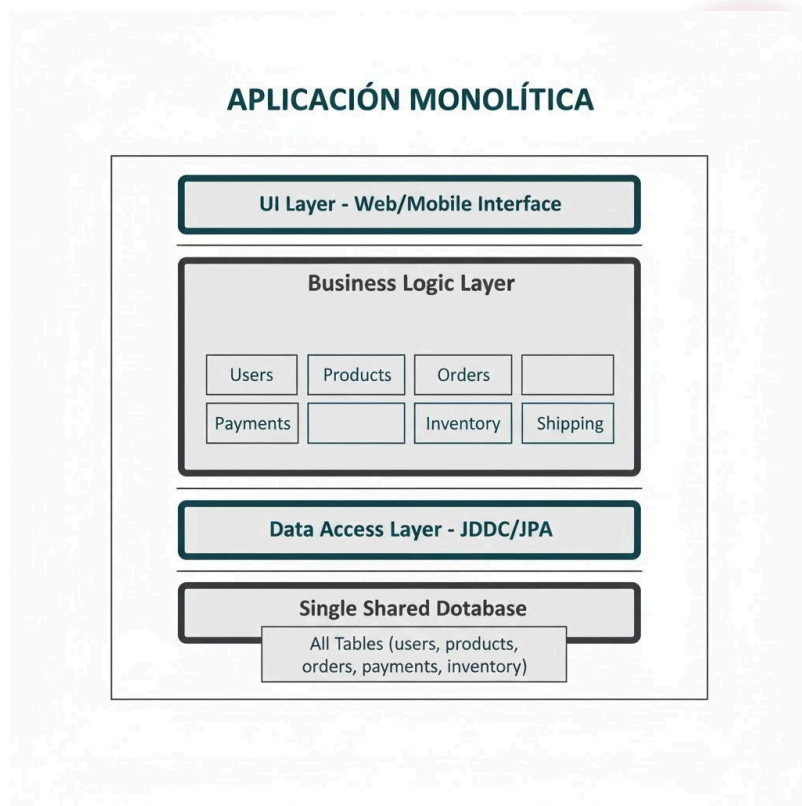
Introducción

La **arquitectura de microservicios** representa un cambio fundamental en cómo diseñamos aplicaciones empresariales. En lugar de construir una aplicación como una unidad monolítica única, la dividimos en un conjunto de servicios pequeños e independientes que trabajan en conjunto.

La Arquitectura Monolítica

Concepto

Un **monolito** es una aplicación donde todos los componentes (UI, lógica de negocio, acceso a datos) están integrados en una sola unidad ejecutable. Toda la funcionalidad reside en un único proceso y comparte una base de datos común.



Deployment: JAR/WAR único
Proceso: Una JVM
Escalamiento: Replicar TODO

Características Principales

Despliegue unificado: Todo el sistema se despliega como una sola pieza. Un cambio en cualquier parte requiere redesplegar toda la aplicación.

Base de datos compartida: Todos los módulos acceden a las mismas tablas, lo que facilita transacciones pero crea acoplamiento fuerte.

Stack tecnológico único: La aplicación completa debe usar el mismo lenguaje y framework.

Ventajas del Monolito

Los monolitos ofrecen **simplicidad inicial**. Son fáciles de desarrollar cuando el equipo es pequeño y el dominio es limitado. El debugging es directo porque todo el flujo ocurre en un solo proceso. Las transacciones ACID son simples de implementar y el rendimiento interno es excelente al evitar latencia de red.

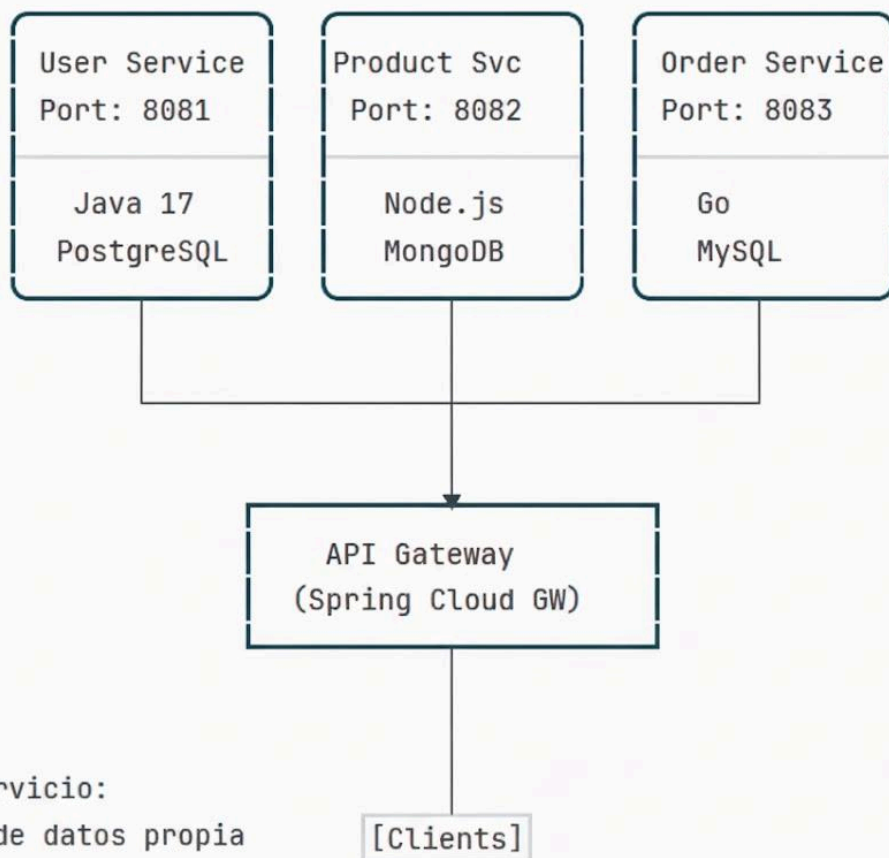
Desventajas del Monolito

Con el tiempo, los monolitos muestran **limitaciones críticas**. El escalamiento es ineficiente porque debes replicar toda la aplicación, incluso si solo una parte necesita más capacidad. Los ciclos de despliegue se vuelven largos y riesgosos. El código se vuelve difícil de mantener conforme crece. Las tecnologías quedan atrapadas en decisiones tempranas. Un error en cualquier módulo puede tumbar todo el sistema.

◆ La Arquitectura de Microservicios

Concepto

Los **microservicios** descomponen la aplicación en servicios pequeños, cada uno ejecutándose en su propio proceso y comunicándose mediante protocolos ligeros (HTTP, mensajería). Cada servicio se construye alrededor de una capacidad de negocio específica y puede desplegarse independientemente.



Cada servicio:

- ✓ Base de datos propia
- ✓ Tecnología independiente
- ✓ Deploy independiente
- ✓ Escala independiente
- ✓ Equino dueño

Principios Fundamentales

1. Responsabilidad Única

Cada microservicio debe tener **una razón clara para existir**. Se enfoca en una capacidad de negocio específica: gestión de usuarios, procesamiento de pedidos, facturación, etc. Esta separación permite que equipos diferentes posean servicios diferentes.

2. Autonomía

Los servicios son **independientes** en desarrollo, despliegue y escalamiento. Cada uno puede evolucionar su tecnología sin afectar a otros. Esta autonomía acelera el desarrollo pero requiere coordinación cuidadosa.

3. Base de Datos por Servicio

Cada microservicio gestiona su **propia base de datos**. Esto garantiza desacoplamiento pero complica las transacciones que cruzan múltiples servicios. Los datos se comparten mediante APIs, no acceso directo a bases de datos.

4. Diseñado para Fallar

En sistemas distribuidos, **los fallos son inevitables**. Los microservicios asumen que las llamadas remotas pueden fallar y se diseñan con timeouts, reintentos, circuit breakers y respuestas de fallback.

Comparativa Fundamental

Dimensión	Monolito	Microservicios
Estructura	Una aplicación	Múltiples servicios
Datos	Base de datos compartida	Base por servicio
Deployment	Todo junto	Independiente
Escalamiento	Vertical (toda la app)	Horizontal (por servicio)
Complejidad	Simple inicialmente	Compleja desde inicio
Resiliencia	Fallo global	Fallo aislado
Tecnología	Stack único	Polyglot posible
Team Structure	Equipo único	Equipos autónomos

Comunicación entre Servicios

Comunicación Síncrona

En la comunicación **síncrona**, un servicio espera la respuesta inmediata de otro. Los protocolos comunes incluyen **REST/HTTP** (simple, universal, basado en texto) y **gRPC** (binario, rápido, con contratos estrictos).

```
java
// Order Service llamando a Product Service
@Service
public class OrderService {

    private final WebClient webClient;

    public Mono<Product> getProductInfo(Long productId) {
        return webClient
            .get()
            .uri("http://product-service/api/products/{id}", productId)
            .retrieve()
            .bodyToMono(Product.class)
            .timeout(Duration.ofSeconds(5))
            .onErrorResume(e -> {
                log.error("Product service unavailable", e);
                return Mono.just(Product.getDefault());
            });
    }
}
```

La ventaja es la **simplicidad conceptual**: solicito algo y obtengo respuesta. Sin embargo, crea **acoplamiento temporal** - si el servicio receptor está caído, la llamada falla. Las latencias se acumulan en cadenas de llamadas, y los servicios deben estar disponibles simultáneamente.

Comunicación Asíncrona

La comunicación **asíncrona** utiliza mensajes o eventos. Un servicio publica eventos (OrderCreated, PaymentProcessed) sin esperar respuesta inmediata. Otros servicios se suscriben a eventos de interés.

```
// Publicar evento
@Service
public class OrderService {

    private final RabbitTemplate rabbitTemplate;

    public void createOrder(Order order) {
        orderRepository.save(order);

        // Publicar evento asíncrono
        OrderCreatedEvent event = new OrderCreatedEvent(
            order.getId(),
            order.getUserId(),
            order.getTotal()
        );
        rabbitTemplate.convertAndSend("orders.exchange",
                                     "order.created",
                                     event);
    }
}

// Consumir evento en otro servicio
@Service
public class InventoryService {

    @RabbitListener(queues = "inventory.orders.queue")
    public void handleOrderCreated(OrderCreatedEvent event) {
        inventoryRepository.reserveStock(event.getOrderId());
    }
}
```

Esto proporciona desacoplamiento temporal

- los servicios no necesitan estar disponibles simultáneamente. Mejora la resiliencia porque los mensajes persisten si un servicio está caído. Sin embargo, aumenta la complejidad: el flujo es más difícil de rastrear y debuggear.

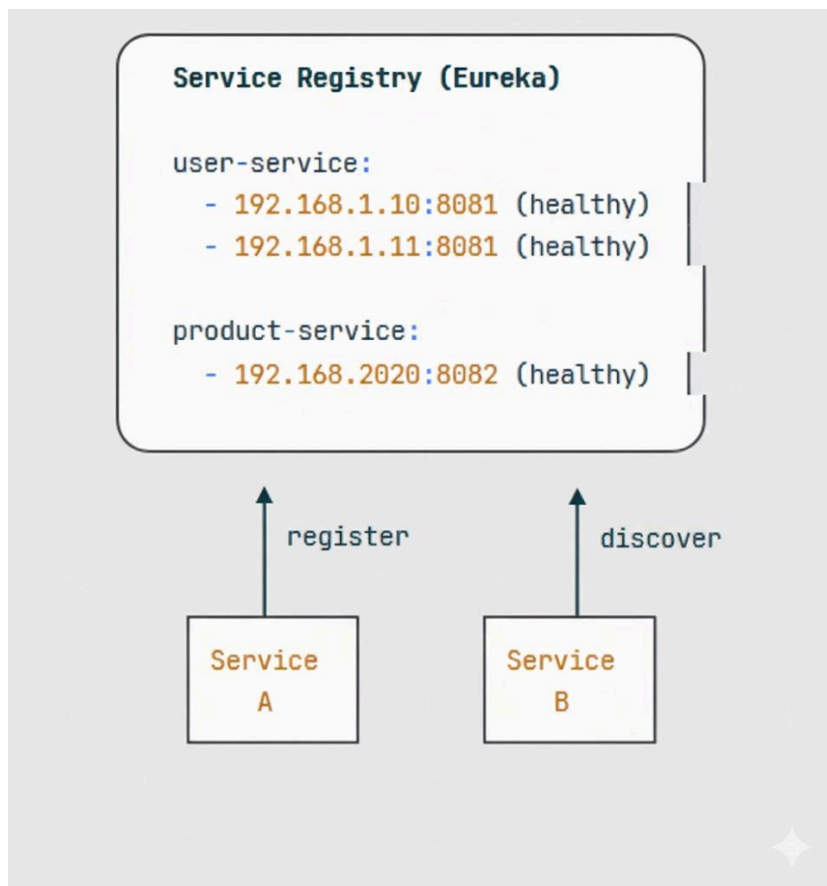
Cuándo usar cada tipo:

- Síncrona: Cuando necesitas respuesta inmediata (consultas, validaciones)
- Asíncrona: Para eventos de negocio, operaciones de larga duración, desacoplamiento

Componentes de Infraestructura

Service Discovery

En entornos dinámicos donde servicios escalan y cambian de ubicación, el descubrimiento de servicios resuelve la pregunta: ¿dónde está el servicio que necesito?



Service Registry actúa como directorio telefónico.

Los servicios se registran al iniciar (nombre, IP, puerto, health check).

Otros servicios consultan el registry para encontrar instancias disponibles. Herramientas populares incluyen **Eureka**, **Consul**, y **etcd**.

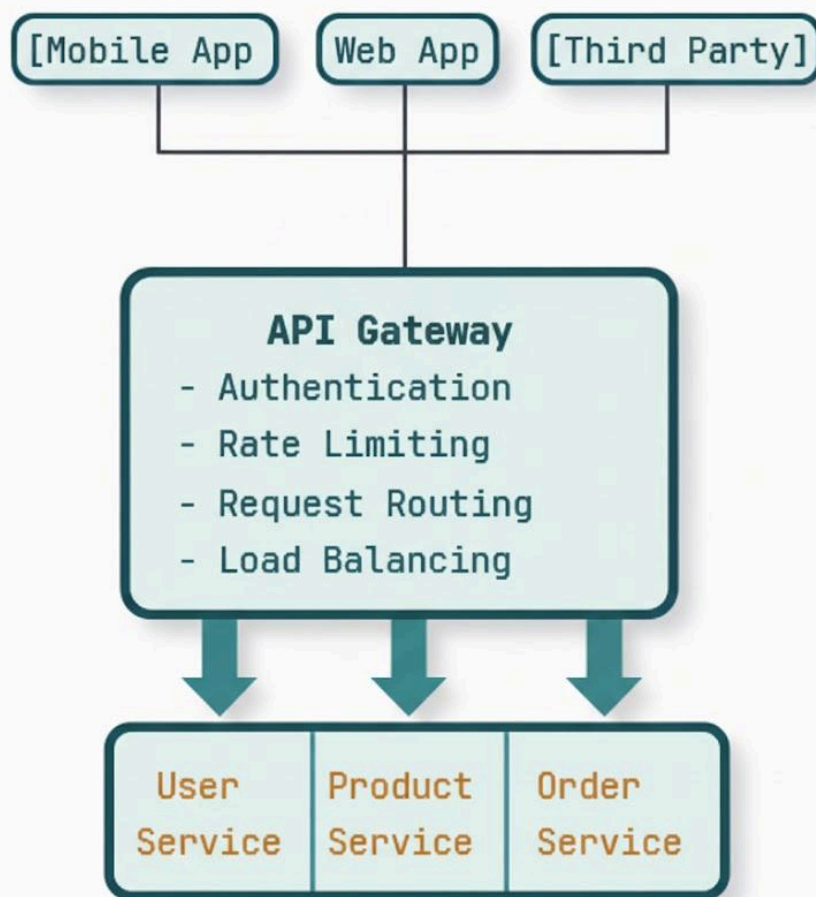
API Gateway

El **API Gateway** es el punto de entrada único para todos los clientes externos. Actúa como fachada, routing requests al servicio apropiado.

Responsabilidades clave:

- **Routing:** Dirigir requests según path/headers
- **Authentication/Authorization:** Validar tokens, permisos
- **Rate Limiting:** Proteger contra abuso
- **Request/Response Transformation:** Agregar headers, modificar formato
- **Caching:** Respuestas frecuentes
- **Logging/Monitoring:** Centralizar observabilidad

El gateway **simplifica** clientes (un solo endpoint) pero se vuelve **punto crítico de fallo** que debe ser altamente disponible.



Patrones de Resiliencia

Circuit Breaker

El **Circuit Breaker** protege contra cascadas de fallos. Funciona como un interruptor eléctrico con tres estados:

CLOSED (Normal): Permite todas las llamadas. Monitorea tasa de fallos. Si excede umbral → OPEN.

OPEN (Circuito Abierto): Rechaza llamadas inmediatamente sin intentar. Retorna respuesta de fallback o error. Después de timeout configurado → HALF-OPEN.

HALF-OPEN (Prueba): Permite algunas llamadas de prueba. Si tienen éxito → CLOSED. Si fallan → OPEN nuevamente.

Este patrón **previene saturación** de servicios ya caídos y permite **recuperación gradual**. La librería **Resilience4j** es estándar en el ecosistema Spring.

Retry Pattern

Los reintentos manejan **fallos transitorios**. Si una llamada falla, espera un tiempo y reintenta. Las estrategias incluyen:

- **Fixed delay:** Espera constante entre reintentos
- **Exponential backoff:** Espera creciente (1s, 2s, 4s, 8s...)
- **Jitter:** Agregar aleatoriedad para evitar thundering herd

Importante: **no reintentar operaciones no idempotentes** (crear pedido, procesar pago) sin precauciones.

Timeout Pattern

Todo llamado remoto debe tener **timeout**. Sin timeout, un servicio lento puede consumir todos los threads esperando indefinidamente. Los timeouts deben ser:

- **Realistas:** Basados en SLAs del servicio
- **Consistentes:** Propagados en la cadena de llamadas
- **Monitoreados:** Alertar si timeouts frecuentes

Gestión de Datos Distribuidos

El Desafío

En monolitos, las transacciones ACID garantizan consistencia. En microservicios, los datos están **distribuidos** - no hay transacciones ACID globales. Esto crea el reto de mantener consistencia sin bloqueos distribuidos.

Database per Service

Cada servicio posee su base de datos, que otros servicios **no pueden acceder directamente**. Los beneficios incluyen autonomía completa, tecnología óptima por caso, y evolución independiente de schemas.

Los desafíos son: ¿cómo hacer queries que cruzan servicios? ¿Cómo mantener consistencia en operaciones multi-servicio?

Saga Pattern

Las **Sagas** coordinan transacciones distribuidas como secuencia de transacciones locales. Si una falla, se ejecutan **transacciones compensatorias** para deshacer cambios anteriores.

Ejemplo: Crear orden implica:

1. Order Service: Crear orden
2. Payment Service: Cargar pago
3. Inventory Service: Reservar stock
4. Shipping Service: Programar envío

Si el paso 3 falla, se compensan pasos 2 y 1 (reembolsar, cancelar orden).

Implementaciones:

- **Choreography:** Cada servicio escucha eventos y decide su acción. Descentralizado pero difícil de rastrear.
- **Orchestration:** Un orquestador coordina la saga. Centralizado, más fácil de debuggear, pero punto de fallo único.

Event Sourcing

En lugar de almacenar estado actual, **Event Sourcing** almacena eventos que llevan a ese estado. El estado se reconstruye aplicando eventos secuencialmente.

Beneficios:

- Auditoría completa (toda la historia)
- Debugging (reproducir estado)
- Proyecciones múltiples del mismo evento stream

Complejidad:

- Reconstrucción puede ser costosa
- Schema evolution de eventos
- Eventual consistency

CQRS (Command Query Responsibility Segregation)

Separar modelos de **escritura** (commands) y **lectura** (queries). Las escrituras actualizan el modelo transaccional. Los eventos generados actualizan modelos de lectura optimizados.

Permite escalar lecturas y escrituras independientemente, pero aumenta complejidad operacional.

Observabilidad Distribuida

El Desafío

En monolitos, el debugging es relativamente simple: logs centralizados, stack traces completos. En microservicios, una request puede atravesar 10+ servicios. ¿Cómo rastrear el flujo completo?

Distributed Tracing

El **tracing distribuido** asigna un **Trace ID** único a cada request. Conforme la request fluye por servicios, cada uno registra un **Span** con el mismo Trace ID. Esto permite visualizar la ruta completa con latencias por servicio.

Componentes:

- **Trace ID:** Identificador único de request end-to-end
- **Span ID:** Identificador de operación en un servicio
- **Parent Span ID:** Para construir jerarquía

Herramientas como **Zipkin** y **Jaeger** visualizan traces completos, facilitando identificar cuellos de botella y fallos.

Logging Centralizado

Los logs deben agregarse en un **almacén centralizado** (ELK Stack: Elasticsearch, Logstash, Kibana). Cada log incluye Trace ID para correlacionar con requests específicos.

Métricas y Health Checks

Cada servicio expone:

- **Métricas:** Latencias, throughput, tasa de errores
- **Health endpoint:** ¿Está el servicio funcionando?

Herramientas de monitoreo (Prometheus, Grafana) agregan métricas y alertan sobre anomalías.

Estrategias de Migración

¿Cuándo Migrar?

Considerar microservicios cuando:

- El monolito es **demasiado grande** para un equipo
- Necesitas **escalar partes específicas**
- Diferentes partes requieren **tecnologías distintas**
- Quieres **releases independientes**

No migrar si:

- Startup o producto nuevo
- Equipo pequeño (< 10 personas)
- Sin expertise en sistemas distribuidos
- Infraestructura DevOps inmadura

Strangler Fig Pattern

La migración **gradual** es más segura que "big bang". El patrón Strangler Fig extrae funcionalidad poco a poco:

1. Identificar **bounded context** para extraer
2. Crear nuevo microservicio
3. Routing configurable (enviar % tráfico al nuevo servicio)
4. Aumentar tráfico gradualmente
5. Una vez estable, remover código del monolito

Este enfoque permite **validar** cada migración antes de continuar y facilita **rollback** si hay problemas.

Anti-Corruption Layer

Al extraer un servicio, puede necesitar comunicarse con el monolito. Un **ACL (Anti-Corruption Layer)** traduce entre los modelos de ambos, previniendo que conceptos del monolito contaminen el diseño limpio del microservicio.

Desafíos y Consideraciones

Complejidad Operacional

Los microservicios aumentan dramáticamente la complejidad operacional. Debes gestionar:



- Múltiples deployments
- Service discovery
- Load balancing
- Distributed tracing
- Centralized logging
- Secret management
- Network policies

Esto requiere **madurez DevOps** con CI/CD robusto, containerización (Docker), orquestación (Kubernetes), y automatización extensiva.

Data Consistency

La **eventual consistency** es el modelo normal. Los datos pueden estar temporalmente inconsistentes entre servicios. Diseñar para esto requiere pensar diferente sobre transacciones y validaciones.

Testing

El testing se complica:

- **Unit tests:** Por servicio (simple)
- **Integration tests:** Requieren múltiples servicios corriendo
- **End-to-end tests:** Costosos, frágiles, lentos
- **Contract testing:** Validar que servicios cumplen contratos acordados

Network Reliability

En monolitos, las llamadas son in-process (memoria). En microservicios, son network calls (HTTP, gRPC). La red **no es confiable**: latencia variable, paquetes perdidos, timeouts. Todo código debe asumir que las llamadas pueden fallar.

5.5 Orquestando Microservicios con Spring Cloud

Spring Cloud es un ecosistema de herramientas construido sobre Spring Boot que simplifica el desarrollo de sistemas distribuidos y microservicios. Proporciona soluciones integradas para los desafíos comunes de arquitecturas distribuidas: configuración, descubrimiento de servicios, circuit breakers, API gateways, y más.

Spring Cloud = Spring Boot + Patrones de Microservicios

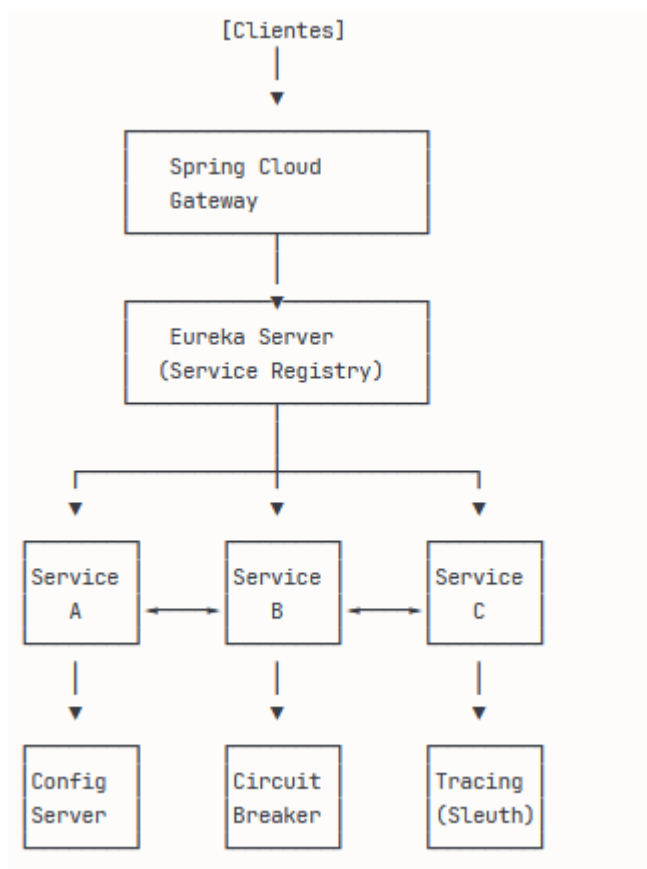
El Ecosistema Spring Cloud

Componentes Principales

Spring Cloud no es una sola herramienta, sino una **colección de proyectos** que resuelven problemas específicos:

Componente	Propósito	Problema que Resuelve
Eureka	Service Discovery	¿Dónde están mis servicios?
Config Server	Configuración Centralizada	¿Cómo gestionar configs?
Gateway	API Gateway	¿Cómo unificar punto de entrada?
Resilience4j	Circuit Breaker	¿Cómo manejar fallos?
Sleuth	Distributed Tracing	¿Cómo rastrear requests?
Stream	Event-Driven	¿Cómo comunicar asincrónamente?
OpenFeign	HTTP Client	¿Cómo simplificar llamadas REST?
LoadBalancer	Client-side LB	¿Cómo distribuir carga?

Arquitectura Típica



Service Discovery con Eureka

El Problema

En arquitecturas de microservicios, los servicios se despliegan dinámicamente en múltiples instancias que pueden cambiar de ubicación (IP, puerto) frecuentemente. Hardcodear direcciones es imposible. Necesitamos un mecanismo para que los servicios se **descubran automáticamente**.

La Solución: Netflix Eureka

Eureka actúa como un **registro centralizado** donde los servicios:

1. Se **registran** al iniciar (nombre, IP, puerto, health status)
2. Envían **heartbeats** periódicos para confirmar que siguen vivos
3. **Consultan** para encontrar otros servicios

Flujo de Service Discovery

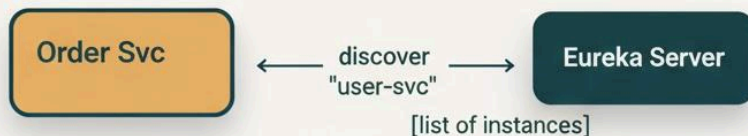
1. Service Registration



2. Heartbeats



3. Service Discovery



Configuración del Eureka Server

El servidor Eureka es extremadamente simple de configurar:

```
java

@SpringBootApplication
@EnableEurekaServer
public class EurekaServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class, args);
    }
}
```

```
properties

# application.properties
server.port=8761
spring.application.name=eureka-server

# No registrarse a sí mismo
eureka.client.register-with-eureka=false
eureka.client.fetch-registry=false

# Dashboard disponible en http://localhost:8761
```

Registrar Microservicios como Clientes

Cada microservicio se convierte en cliente de Eureka simplemente agregando una anotación:

```
java
@SpringBootApplication
@EnableDiscoveryClient
public class UserServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(UserServiceApplication.class, args);
    }
}
```

```
properties

# application.properties
spring.application.name=user-service
server.port=8081

# URL del Eureka Server
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/

# Configuración de heartbeat
eureka.instance.lease-renewal-interval-in-seconds=30
eureka.instance.lease-expiration-duration-in-seconds=90
...
```

Características Clave

****Self-Preservation Mode:**** Si muchos servicios dejan de enviar heartbeats simultáneamente, Eureka asume un problema de red y NO los desregistra, evitando eliminar servicios que realmente están vivos.

****Multi-Zone Awareness:**** Eureka puede desplegar réplicas en múltiples zonas para alta disponibilidad.

****Client-side Caching:**** Los clientes cachean el registro localmente, continuando operando incluso si Eureka cae temporalmente.

API Gateway con Spring Cloud Gateway

Sin un gateway, los clientes deben conocer y llamar a ****múltiples microservicios**** directamente. Esto resulta en:

- Múltiples puntos de entrada (complejidad)
- Cross-cutting concerns duplicados (auth, logging, rate limiting)

- Acoplamiento entre clientes y estructura interna
- Dificultad para refactorizar servicios

La Solución: Spring Cloud Gateway

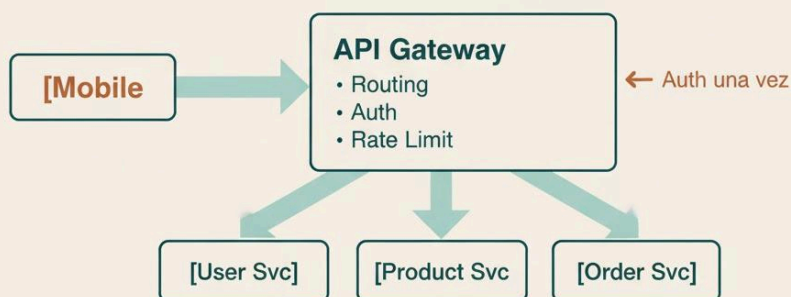
El **API Gateway** es el **único punto de entrada** que:

- Enruta requests al servicio apropiado
- Centraliza autenticación, autorización, rate limiting
- Transforma requests y responses
- Implementa circuit breakers y retries
- Oculta la estructura interna de microservicios

Sin Gateway:



Con Gateway:



Event-Driven con Spring Cloud Stream

El Problema

La comunicación síncrona (REST) crea **acoplamiento temporal**. Para eventos de negocio y operaciones asíncronas, necesitamos mensajería.

Spring Cloud Stream

Spring Cloud Stream abstrae sistemas de mensajería (RabbitMQ, Kafka) con una API unificada basada en **functional programming**.

```
java
@Configuration
public class OrderEvents {

    // Publicar eventos
    @Bean
    public Supplier<Flux<OrderCreatedEvent>> orderPublisher() {
        return () -> Flux.interval(Duration.ofSeconds(1))
            .map(i -> new OrderCreatedEvent(i, "user-" + i));
    }

    // Consumir eventos
    @Bean
    public Consumer<OrderCreatedEvent> orderConsumer() {
        return event -> {
            log.info("Processing order: {}", event.getOrderId());
            inventoryService.reserve(event.getOrderId());
        };
    }

    // Transformar eventos (processor)
    @Bean
    public Function<OrderCreatedEvent, OrderProcessedEvent> orderProcessor() {
        return orderCreated -> {
            // Procesar lógica
            return new OrderProcessedEvent(orderCreated.getOrderId());
        };
    }
}
```

configuracion

```
yaml

spring:
  cloud:
    stream:
      bindings:
        # Output (publicar)
        orderPublisher-out-0:
          destination: orders

        # Input (consumir)
        orderConsumer-in-0:
          destination: orders
          group: inventory-service

        # Processor
        orderProcessor-in-0:
          destination: orders
        orderProcessor-out-0:
          destination: orders-processed
```

Monitoreo con Spring Boot Actuator

Actuator Endpoints

Cada microservicio debe exponer métricas y health checks:

```
management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus

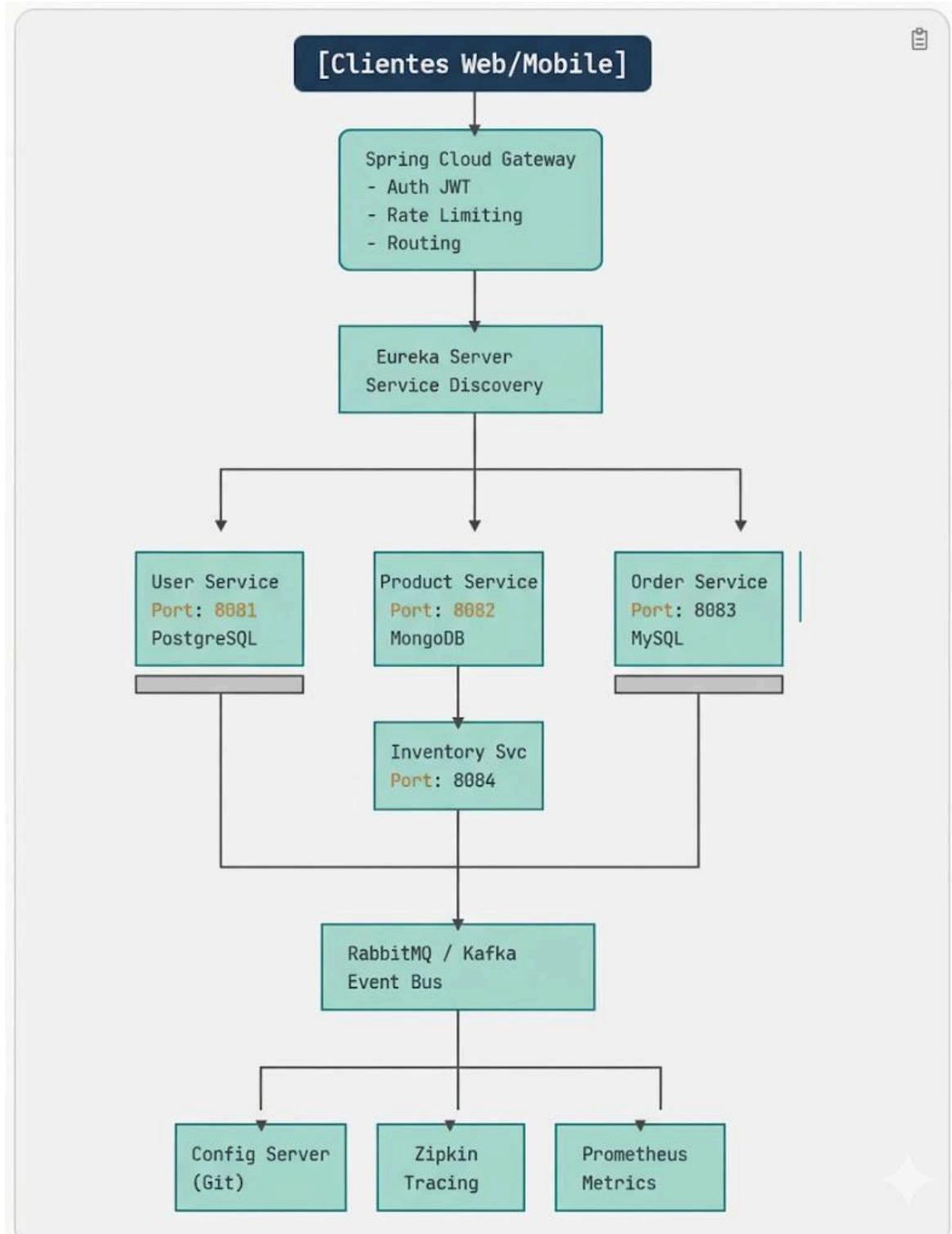
  endpoint:
    health:
      show-details: always

  health:
    circuitbreakers:
      enabled: true
    ratelimiters:
      enabled: true
```


****Endpoints útiles:****

- `/actuator/health` - Estado del servicio y dependencias
- `/actuator/metrics` - Métricas de rendimiento
- `/actuator/prometheus` - Formato Prometheus
- `/actuator/info` - Información del servicio
- `/actuator/env` - Variables de entorno





Clientes (Web/Móvil)

Envían todas las solicitudes al sistema.

Spring Cloud Gateway

- Revisa tokens JWT
- Controla límites de uso
- Decide a qué microservicio enviar cada petición

Eureka Server (Discovery)

Mantiene un “mapa” de todos los microservicios y sus puertos, para que el Gateway sepa dónde enviarlos.

Microservicios principales

- **User Service (8081 – PostgreSQL)**
- **Product Service (8082 – MongoDB)**
- **Order Service (8083 – MySQL)**

Cada uno maneja su propia lógica y base de datos.

Inventory Service (8084)

Conecta información de productos y órdenes para mantener el inventario actualizado.

RabbitMQ / Kafka (Event Bus)

Permite que los microservicios se comuniquen enviando eventos sin

depender unos de otros.

Servicios de apoyo

- **Config Server:** configura todos los servicios desde un solo lugar.
- **Zipkin:** rastrea peticiones entre microservicios.
- **Prometheus:** monitorea métricas del sistema.

El sistema usa un Gateway + Eureka para dirigir peticiones a microservicios independientes que se comunican por eventos y se monitorean con herramientas externas.

